PROCESSO SELETIVO CITI · DESAFIO DE BACK-END

🌿 Folhagem & Cia

Construa a API de uma loja de plantas: um CRUD completo com regras de negócio de verdade.

Node.js | Express | TypeScript | Prisma | PostgreSQL | Docker

A história

A **Folhagem & Cia** vende plantas pela internet e controla tudo numa planilha. Já deu problema: a loja vendeu uma samambaia que não existia mais no estoque. 😬 A sua missão é construir a **API** que substitui essa planilha.

> 💡 **Você não começa do zero**
>
> O repositório já tem um **CRUD de usuários completo e funcionando** (`src/controllers/UserController.ts` e `src/repositories/UserRepository.ts`). Ele é o seu modelo: quase tudo que você vai fazer nas plantas tem um equivalente lá. Leia, rode e entenda antes de começar.

1 O que construir

A entidade `Planta`

Campo	Tipo	Exemplo	Observação
`id`	Int (autoincrement)	`1`	Gerado pelo banco
`nome`	String	`"Costela-de-adão"`	Nome popular
`especie`	String	`"Monstera deliciosa"`	Nome científico
`preco`	Int	`8990`	Em **centavos**: `8990` = R$ 89,90
`estoque`	Int	`12`	Vasos disponíveis
`luminosidade`	Enum	`"MEIA_SOMBRA"`	`SOMBRA` , `MEIA_SOMBRA` ou `SOL_PLENO`
`petFriendly`	Boolean	`true`	Segura para cães e gatos?
`regaACadaDias`	Int	`3`	De quantos em quantos dias regar
`createdAt` / `updatedAt`	DateTime		Automáticos, como no `User`

Por que centavos? Decimais têm erros de arredondamento (`0.1 + 0.2` dá `0.30000000000000004`). Com inteiros, as contas com dinheiro são sempre exatas.

Rotas

#	Rota	O que faz	Sucesso
1	`POST` `/plantas`	Cadastra uma planta	`201`
2	`GET` `/plantas`	Lista as plantas, com filtros opcionais `?luminosidade=` e `?petFriendly=`, sozinhos ou juntos	`200`
3	`GET` `/plantas/:id`	Busca uma planta	`200`
4	`PATCH` `/plantas/:id`	Edita só os campos enviados	`200`
5	`DELETE` `/plantas/:id`	Remove uma planta	`204`
6	`POST` `/plantas/:id/venda`	Recebe `{ "quantidade": 2 }` e diminui o estoque	`200`
7	`GET` `/plantas/estoque/valor-total`	Soma `preco × estoque` de todas as plantas e o total de vasos: `{ "valorTotalEmCentavos": 154300, "totalDeVasos": 37 }`	`200`

Regras de negócio

1. Todos os campos são obrigatórios no cadastro.
2. `preco` maior que zero; `regaACadaDias` pelo menos `1`.
3. `estoque` não pode ser negativo, mas **pode ser zero** (planta esgotada).
4. `luminosidade` só aceita os três valores do enum.
5. Dados inválidos (no cadastro, na edição, nos filtros ou na venda) retornam `400` com uma mensagem clara.
6. Um `id` que não existe retorna `404`.
7. A venda **nunca** deixa o estoque negativo: sem estoque suficiente, retorna `400` e não altera nada.

Bônus (só depois do obrigatório)

- Busca por nome sem diferenciar maiúsculas: `?nome=costela` · Ordenação: `?ordem=preco` ou `?ordem=nome`
- Rota `GET /plantas/esgotadas` · Script de *seed* com plantas de exemplo

2 Rodando o projeto

Você precisa de Git, Node.js 20+, Yarn (`npm install -g yarn`), Docker Desktop e um cliente HTTP (Insomnia, Postman ou a extensão Thunder Client do VS Code).

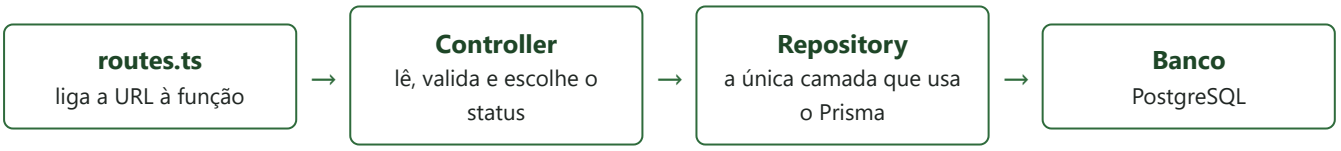
```
git clone <link-do-repositório> && cd treinamento-crud-citi
git checkout -b desafio/seu-nome
cp .env.example .env              # no PowerShell: copy .env.example .env
yarn
docker compose up -d --build      # com o Docker Desktop aberto
yarn migration                    # cria as tabelas; dê um nome, ex.: create_user_model
```

Abra `http://localhost:3001`: deve aparecer a mensagem da API. A API reinicia sozinha quando você salva um arquivo. Para ver erros e logs: `docker compose logs -f server`. Para ver o banco: `yarn studio`.

Se aparecer...	Faça
`failed to connect to the docker API`	Abra o Docker Desktop e espere ele ficar pronto.
`port is already allocated`	Troque `SERVER_PORT` ou `DATABASE_PORT` no `.env` e suba de novo.
`Can't reach database server`	Os containers não estão de pé: `docker compose up -d`.
`Property 'planta' does not exist`	Rode `yarn migration` e reinicie o TypeScript no VS Code (`Ctrl+Shift+P` → *Restart TS Server*).
`req.body` chega `undefined`	No cliente HTTP, envie o corpo como **JSON**.

3 Como construir

O projeto separa o código em camadas. Siga a mesma organização do CRUD de usuário:



Siga os passos na ordem e **faça um commit ao fim de cada um**.

1 Model e migration

Em `prisma/schema.prisma`, crie o `enum Luminosidade` e o `model Planta`. Depois rode `yarn migration` com o nome `create_planta_model`. A sintaxe do enum:

```
enum Tamanho {
  PEQUENO
  MEDIO
  GRANDE
}
```

No model, ele é usado como tipo: `tamanho Tamanho`. A migration gerada vai para o git junto com o código. Se errar o model, corrija o schema e rode outra migration: nunca edite uma migration já aplicada.

2 Repository

Crie `src/repositories/PlantaRepository.ts` com uma função por consulta (`createPlanta`, `findAllPlantas`, `findPlantaById`, `updatePlanta`, `deletePlanta`...), como no `UserRepository`. Diferente do usuário, **o id da planta é número**: as funções recebem `id: number`.

3 Controller

Crie `src/controllers/PlantaController.ts`. Cada handler segue o roteiro do `createUser`: **ler** → **validar** → **checar regras** → **chamar o repository** → **responder**, tudo dentro de um `try/catch` que responde `500` em erro inesperado. Os detalhes que mais derrubam candidatos:

> 🚫 **Não valide com `!campo`**
>
> `!0` e `!false` são `true`. Então `if (!estoque)` recusa uma planta com estoque zero, e `if (!petFriendly)` recusa toda planta que não é pet-friendly, e as duas são válidas. Para saber se o campo foi enviado, use `campo === undefined`.

Para validar...	Use
Texto não vazio	`typeof nome === "string" && nome.trim() !== ""`
Inteiro (recusa `"8990"`, `89.9` e `NaN`)	`Number.isInteger(preco)`
Booleano	`typeof petFriendly === "boolean"`
Valor do enum	`Object.values(Luminosidade).includes(luminosidade)`, com `import { Luminosidade } from "@prisma/client"`
O `:id` da rota (chega como texto)	`const id = Number(req.params.id)` e confira com `Number.isInteger(id) && id > 0`

- **POST x PATCH:** no cadastro, todos os campos são obrigatórios. Na edição, nenhum é, mas os que vierem passam pelas mesmas regras. Uma função que valida só os campos enviados serve para os dois.
- **404 antes de editar ou deletar:** busque a planta primeiro e responda `404` se não existir. Sem isso, o erro do Prisma cai no `catch` e vira um 500 errado.
- **Não passe `req.body` direto para o banco:** desestruture e envie só os campos da planta.
- **Todo `res.status(...)` dentro de um `if` precisa de `return`.** Sem ele, a função continua e quebra com "*Cannot set headers after they are sent*".
- **Mensagens com variáveis usam crase:** `` `Temos só ${planta.estoque} vasos` ``. Com aspas duplas, o texto sai literal.

4 Rotas

Registre as rotas em `src/routes.ts`. O Express usa a **primeira rota que combinar**, então rotas fixas vêm antes das que têm parâmetro: `/plantas/esgotadas` precisa ser registrada antes de `/plantas/:id`, senão "esgotadas" é tratado como um id.

5 Filtros

- Os filtros chegam em `req.query` e são **sempre texto**: `?petFriendly=true` chega como `"true"`. Converta com `=== "true"`, mas sem transformar um filtro não enviado (`undefined`) em `false`.

- No `where` do Prisma, condições com valor `undefined` são ignoradas. Uma única consulta resolve todas as combinações de filtros.
- Filtro com valor inválido (`?luminosidade=LUA`) é `400`. Filtro válido sem resultado é `200` com `[]`, não 404.

6 Venda

Ordem do handler: valide o id e a `quantidade` (inteiro maior que zero) → busque a planta (`404`) → confira `estoque >= quantidade` (senão `400`, sem alterar nada) → diminua o estoque → responda `200` com a planta atualizada. O Prisma decrementa sozinho: `data: { estoque: { decrement: quantidade } }`.

Para pensar: e se duas pessoas comprarem o último vaso ao mesmo tempo? Se você resolver isso, conte na entrega.

7 Valor total do estoque

O `aggregate` do Prisma soma uma coluna, mas não soma `preco × estoque`. Busque as plantas (só esses dois campos, com `select`) e faça a conta no código, com um `for` ou `reduce`. Com a loja vazia, a resposta é `200` com os valores zerados.

4 Testes essenciais

Teste no cliente HTTP, mandando os tipos certos no JSON (`"preco": 8990`, sem aspas). **A maior parte da avaliação está nos casos de erro.**

Teste	Esperado
Cadastrar uma planta válida, inclusive com `estoque: 0` e `petFriendly: false`	`201`
Cadastrar sem um campo, com `preco: 0`, `estoque: -1`, `preco: "8990"` ou `luminosidade: "LUA"`	`400`
Buscar um id que existe / que não existe / `abc`	`200` / `404` / `400`
Editar só o preço (o resto não muda) / editar com valor inválido / id inexistente	`200` / `400` / `404`
Filtrar por luminosidade, por petFriendly e pelos dois / filtro inválido	`200` / `400`
Vender o que tem / vender mais do que tem (o estoque não pode mudar) / `quantidade: 0`	`200` / `400` / `400`
Valor total (confira a conta na mão)	`200`
Deletar / buscar a planta deletada / deletar de novo	`204` / `404` / `404`

5 Entrega e avaliação

Antes de entregar

- ☐ Sobe do zero com `docker compose up` + `yarn migration`
- ☐ Migration da `Planta` commitada
- ☐ Nomes de campos e rotas iguais aos do enunciado
- ☐ Todos os testes da parte 4 passam
- ☐ `yarn typecheck` sem erros
- ☐ Sem `console.log` de teste, imports sem uso ou código comentado
- ☐ Commits pequenos e com mensagens claras
- ☐ `.env` fora do git

Como vamos avaliar

Funcionamento	Todas as rotas funcionam
Status HTTP	O status certo em cada caso
Validações	Regras respeitadas, mensagens claras
Organização	Controller e repository separados
Prisma	Model, migration, `where` e enum
Código limpo	Nomes claros, sem código morto

Para entregar: suba a branch `desafio/seu-nome` e envie o link do repositório pelo canal e no prazo informados pelos instrutores. Se fez bônus ou tomou alguma decisão diferente, explique no README.

Os bônus contam pontos extras, mas não compensam requisitos obrigatórios faltando. Travou? Leia o erro nos logs, compare com o CRUD de usuário e consulte prisma.io/docs. Se continuar travado, pergunte aos instrutores, dizendo o que tentou e qual erro apareceu.

> 🌿 **Boa sorte e boas regas!**
>
> Faça com calma, teste cada passo e capriche nos casos de erro. É neles que uma API mostra que foi bem feita.